DATA STRUCTURES LABORATORY

Assignments 1–11 | Assignments 1–3 in Python | Assignments 4–11 in C++

Simple programs with sample expected outputs

Assignment 1

Python Code

```
students = [
    [101, "Aarav", 8.7],
    [105, "Diya", 9.4],
    [103, "Kabir", 8.9],
    [102, "Isha", 9.8],
    [110, "Neha", 8.5],
    [107, "Om", 9.6],
    [104, "Riya", 9.1],
    [109, "Vivaan", 8.8],
    [106, "Mira", 9.3],
    [108, "Arjun", 9.0]
]

def insertion_sort(data, key):
    result = data[:]
    for i in range(1, len(result)):
        current = result[i]
        j = i - 1
        while j >= 0 and result[j][key] > current[key]:
            result[j + 1] = result[j]
            j -= 1
        result[j + 1] = current
    return result

print("Students by roll number:")
for s in insertion_sort(students, 0):
    print(s)

print("\\nStudents alphabetically:")
for s in insertion_sort(students, 1):
    print(s)

print("\\nFive toppers:")
for s in insertion_sort(students, 2)[-5:][::-1]:
    print(s)

employees = [
    [205, "Karan", 35000], [201, "Aditi", 42000],
    [208, "Zoya", 39000], [203, "Manav", 31000],
    [206, "Bhavna", 45000], [202, "Rohan", 37000],
    [210, "Tara", 48000], [204, "Ira", 33000],
    [209, "Dev", 41000], [207, "Nisha", 36000]
]

def quick_sort(data, key):
    if len(data) <= 1:
        return data
    pivot = data[len(data) // 2]
    left = [x for x in data if x[key] < pivot[key]]
    middle = [x for x in data if x[key] == pivot[key]]
    right = [x for x in data if x[key] > pivot[key]]
    return quick_sort(left, key) + middle + quick_sort(right, key)

def merge_sort(data, key):
    if len(data) <= 1:
        return data
    mid = len(data) // 2
    left = merge_sort(data[:mid], key)
    right = merge_sort(data[mid:], key)
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i][key].lower() <= right[j][key].lower():
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    return result + left[i:] + right[j:]

print("\\nEmployees by ID using Quick Sort:")
for e in quick_sort(employees, 0):
    print(e)

print("\\nEmployees alphabetically using Merge Sort:")
for e in merge_sort(employees, 1):
    print(e)
```

Expected Output

```
Students by roll number:
```

```
[101, 'Aarav', 8.7]
[102, 'Isha', 9.8]
[103, 'Kabir', 8.9]
[104, 'Riya', 9.1]
[105, 'Diya', 9.4]
[106, 'Mira', 9.3]
[107, 'Om', 9.6]
[108, 'Arjun', 9.0]
[109, 'Vivaan', 8.8]
[110, 'Neha', 8.5]

Students alphabetically:
[101, 'Aarav', 8.7]
[108, 'Arjun', 9.0]
[105, 'Diya', 9.4]
[102, 'Isha', 9.8]
[103, 'Kabir', 8.9]
[106, 'Mira', 9.3]
[110, 'Neha', 8.5]
[107, 'Om', 9.6]
[104, 'Riya', 9.1]
[109, 'Vivaan', 8.8]

Five toppers:
[102, 'Isha', 9.8]
[107, 'Om', 9.6]
[105, 'Diya', 9.4]
[106, 'Mira', 9.3]
[104, 'Riya', 9.1]

Employees by ID using Quick Sort:
[201, 'Aditi', 42000]
[202, 'Rohan', 37000]
[203, 'Manav', 31000]
[204, 'Ira', 33000]
[205, 'Karan', 35000]
[206, 'Bhavna', 45000]
[207, 'Nisha', 36000]
[208, 'Zoya', 39000]
[209, 'Dev', 41000]
[210, 'Tara', 48000]

Employees alphabetically using Merge Sort:
[206, 'Bhavna', 45000]
[209, 'Dev', 41000]
[204, 'Ira', 33000]
[205, 'Karan', 35000]
[203, 'Manav', 31000]
[207, 'Nisha', 36000]
[201, 'Aditi', 42000]
[202, 'Rohan', 37000]
[210, 'Tara', 48000]
[208, 'Zoya', 39000]
```

Assignment 2

Python Code

```
books = [
    [1, "Python Basics"], [2, "Data Structures"],
    [3, "Computer Networks"], [4, "Database Systems"],
    [5, "Operating Systems"], [6, "Java Programming"],
    [7, "Web Development"], [8, "Digital Logic"],
    [9, "Machine Learning"], [10, "Software Engineering"]
]

def linear_search(data, target):
    comparisons = 0
    for book in data:
        comparisons += 1
        if book[1].lower() == target.lower():
            return book, comparisons
    return None, comparisons

def binary_search(data, target):
    low = 0
    high = len(data) - 1
    comparisons = 0
    while low <= high:
        mid = (low + high) // 2
        comparisons += 1
        name = data[mid][1].lower()
        if name == target.lower():
            return data[mid], comparisons
        if name < target.lower():
            low = mid + 1
        else:
            high = mid - 1
    return None, comparisons

title = "Machine Learning"
found, count = linear_search(books, title)
print("Linear Search:", found, "Comparisons:", count)

sorted_books = sorted(books, key=lambda x: x[1].lower())
found, count = binary_search(sorted_books, title)
print("Binary Search:", found, "Comparisons:", count)
```

Expected Output

```
Linear Search: [9, 'Machine Learning'] Comparisons: 9
Binary Search: [9, 'Machine Learning'] Comparisons: 1
```

Assignment 3

Python Code

```
class HashTable:
    def __init__(self, size):
        self.size = size
        self.linear = [None] * size
        self.chain = [[] for _ in range(size)]

    def hash_function(self, emp_id):
        return emp_id % self.size

    def insert_linear(self, emp_id, name):
        index = self.hash_function(emp_id)
        start = index
        while self.linear[index] is not None:
            index = (index + 1) % self.size
            if index == start:
                print("Linear table is full")
                return
        self.linear[index] = (emp_id, name)

    def insert_chain(self, emp_id, name):
        index = self.hash_function(emp_id)
        self.chain[index].append((emp_id, name))

    def display(self):
        print("Linear Probing:")
        for i, item in enumerate(self.linear):
            print(i, ":", item)
        print("Chaining:")
        for i, items in enumerate(self.chain):
            print(i, ":", items)

records = [
    (101, "Asha"), (111, "Rohan"), (121, "Meera"),
    (102, "Kabir"), (112, "Diya"), (103, "Om")
]

table = HashTable(10)
for emp_id, name in records:
    table.insert_linear(emp_id, name)
    table.insert_chain(emp_id, name)

table.display()
```

Expected Output

```
Linear Probing:
0 : None
1 : (101, 'Asha')
2 : (102, 'Kabir')
3 : (103, 'Om')
4 : None
5 : None
6 : None
7 : None
8 : None
9 : None
Chaining:
0 : []
1 : [(101, 'Asha'), (111, 'Rohan'), (121, 'Meera')]
2 : [(102, 'Kabir'), (112, 'Diya')]
3 : [(103, 'Om')]
4 : []
5 : []
6 : []
7 : []
8 : []
9 : []
```

Assignment 4

C++ Code

```
#include <iostream>
#include <string>
using namespace std;

struct Node {
    int roll;
    string name;
    float marks;
    Node* next;
};

Node* head = nullptr;

void addStudent(int roll, string name, float marks) {
    Node* n = new Node{roll, name, marks, nullptr};
    if (head == nullptr) {
        head = n;
        return;
    }
    Node* t = head;
    while (t->next != nullptr) t = t->next;
    t->next = n;
}

void deleteStudent(int roll) {
    Node *t = head, *prev = nullptr;
    while (t != nullptr && t->roll != roll) {
        prev = t;
        t = t->next;
    }
    if (t == nullptr) {
        cout << "Student not found\\n";
        return;
    }
    if (prev == nullptr) head = t->next;
    else prev->next = t->next;
    delete t;
    cout << "Record deleted\\n";
}

void searchStudent(int roll) {
    Node* t = head;
    while (t != nullptr) {
        if (t->roll == roll) {
            cout << t->roll << " " << t->name << " " << t->marks << "\\n";
            return;
        }
        t = t->next;
    }
    cout << "Student not found\\n";
}

void display() {
    Node* t = head;
    if (t == nullptr) cout << "List is empty\\n";
    while (t != nullptr) {
        cout << t->roll << " " << t->name << " " << t->marks << "\\n";
        t = t->next;
    }
}

int main() {
    int choice, roll;
    string name;
    float marks;
    do {
        cout << "\\n1.Add 2.Delete 3.Search 4.Display 0.Exit\\n";
        cin >> choice;
        if (choice == 1) {
            cout << "Enter roll, name and marks: ";
            cin >> roll >> name >> marks;
            addStudent(roll, name, marks);
        } else if (choice == 2) {
            cin >> roll;
            deleteStudent(roll);
        } else if (choice == 3) {
            cin >> roll;
            searchStudent(roll);
        } else if (choice == 4) display();
    } while (choice != 0);
    return 0;
}
```

Expected Output

```
1.Add 2.Delete 3.Search 4.Display 0.Exit
1
Enter roll, name and marks: 101 Aarav 85
1.Add 2.Delete 3.Search 4.Display 0.Exit
1
Enter roll, name and marks: 102 Diya 92
```

```
1.Add 2.Delete 3.Search 4.Display 0.Exit
4
101 Aarav 85
102 Diya 92
1.Add 2.Delete 3.Search 4.Display 0.Exit
3
102
102 Diya 92
1.Add 2.Delete 3.Search 4.Display 0.Exit
2
101
Record deleted
```

Assignment 5

C++ Code

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *prev, *next;
};

Node* head = nullptr;

void insertEnd(int value) {
    Node* n = new Node{value, nullptr, nullptr};
    if (head == nullptr) {
        head = n;
        return;
    }
    Node* t = head;
    while (t->next != nullptr) t = t->next;
    t->next = n;
    n->prev = t;
}

void deleteValue(int value) {
    Node* t = head;
    while (t != nullptr && t->data != value) t = t->next;
    if (t == nullptr) {
        cout << "Value not found\\n";
        return;
    }
    if (t->prev != nullptr) t->prev->next = t->next;
    else head = t->next;
    if (t->next != nullptr) t->next->prev = t->prev;
    delete t;
    cout << "Deleted\\n";
}

void searchValue(int value) {
    Node* t = head;
    while (t != nullptr) {
        if (t->data == value) {
            cout << "Found\\n";
            return;
        }
        t = t->next;
    }
    cout << "Not found\\n";
}

void display() {
    Node* t = head;
    while (t != nullptr) {
        cout << t->data << " ";
        t = t->next;
    }
    cout << "\\n";
}

int main() {
    int ch, value;
    do {
        cout << "1.Insert 2.Delete 3.Search 4.Display 0.Exit\\n";
        cin >> ch;
        if (ch == 1) { cin >> value; insertEnd(value); }
        else if (ch == 2) { cin >> value; deleteValue(value); }
        else if (ch == 3) { cin >> value; searchValue(value); }
        else if (ch == 4) display();
    } while (ch != 0);
    return 0;
}
```

Expected Output

```
1.Insert 2.Delete 3.Search 4.Display 0.Exit
1
10
1.Insert 2.Delete 3.Search 4.Display 0.Exit
1
20
1.Insert 2.Delete 3.Search 4.Display 0.Exit
1
30
1.Insert 2.Delete 3.Search 4.Display 0.Exit
4
10 20 30
1.Insert 2.Delete 3.Search 4.Display 0.Exit
3
20
Found
1.Insert 2.Delete 3.Search 4.Display 0.Exit
2
```

```
20
Deleted
```

Assignment 6

C++ Code

```
#include <iostream>
#include <string>
using namespace std;

struct Node {
    string song;
    Node* next;
};

Node* tail = nullptr;

void addSong(string song) {
    Node* n = new Node{song, nullptr};
    if (tail == nullptr) {
        tail = n;
        n->next = n;
    } else {
        n->next = tail->next;
        tail->next = n;
        tail = n;
    }
}

void deleteSong(string song) {
    if (tail == nullptr) {
        cout << "Playlist is empty\\n";
        return;
    }
    Node* current = tail->next;
    Node* previous = tail;
    do {
        if (current->song == song) {
            if (current == tail && current->next == tail) tail = nullptr;
            else {
                previous->next = current->next;
                if (current == tail) tail = previous;
            }
            delete current;
            cout << "Song deleted\\n";
            return;
        }
        previous = current;
        current = current->next;
    } while (current != tail->next);
    cout << "Song not found\\n";
}

void display() {
    if (tail == nullptr) {
        cout << "Playlist is empty\\n";
        return;
    }
    Node* t = tail->next;
    do {
        cout << t->song << " -> ";
        t = t->next;
    } while (t != tail->next);
    cout << "(loop)\\n";
}

int main() {
    int ch;
    string song;
    do {
        cout << "1.Add 2.Delete 3.Display 0.Exit\\n";
        cin >> ch;
        if (ch == 1) { cin >> song; addSong(song); }
        else if (ch == 2) { cin >> song; deleteSong(song); }
        else if (ch == 3) display();
    } while (ch != 0);
    return 0;
}
```

Expected Output

```
1.Add 2.Delete 3.Display 0.Exit
1
Believer
1.Add 2.Delete 3.Display 0.Exit
1
Perfect
1.Add 2.Delete 3.Display 0.Exit
3
Believer -> Perfect -> (loop)
1.Add 2.Delete 3.Display 0.Exit
2
Believer
Song deleted
1.Add 2.Delete 3.Display 0.Exit
3
Perfect -> (loop)
```

Assignment 7

C++ Code

```
#include <iostream>
#include <stack>
#include <string>
#include <cctype>
using namespace std;

int priority(char op) {
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}

string toPostfix(string infix) {
    stack<char> st;
    string result;
    for (char ch : infix) {
        if (isalnum(ch)) result += ch;
        else if (ch == '(') st.push(ch);
        else if (ch == ')') {
            while (!st.empty() && st.top() != '(') {
                result += st.top();
                st.pop();
            }
            if (!st.empty()) st.pop();
        } else {
            while (!st.empty() && priority(st.top()) >= priority(ch)) {
                result += st.top();
                st.pop();
            }
            st.push(ch);
        }
    }
    while (!st.empty()) {
        result += st.top();
        st.pop();
    }
    return result;
}

int evaluate(string postfix) {
    stack<int> st;
    for (char ch : postfix) {
        if (isdigit(ch)) st.push(ch - '0');
        else {
            int b = st.top(); st.pop();
            int a = st.top(); st.pop();
            if (ch == '+') st.push(a + b);
            else if (ch == '-') st.push(a - b);
            else if (ch == '*') st.push(a * b);
            else if (ch == '/') st.push(a / b);
        }
    }
    return st.top();
}

int main() {
    string infix;
    cout << "Enter single-digit infix expression: ";
    cin >> infix;
    string postfix = toPostfix(infix);
    cout << "Postfix: " << postfix << "\\n";
    cout << "Result: " << evaluate(postfix) << "\\n";
    return 0;
}
```

Expected Output

```
Enter single-digit infix expression: 2+3*4
Postfix: 234*+
Result: 14
```

Assignment 8

C++ Code

```
#include <iostream>
#include <queue>
#include <string>
using namespace std;

int main() {
    queue<string> customers;
    int ch;
    string name;
    do {
        cout << "1.Enqueue 2.Dequeue 3.Display 0.Exit\\n";
        cin >> ch;
        if (ch == 1) {
            cout << "Customer name: ";
            cin >> name;
            customers.push(name);
            cout << "Ticket booked\\n";
        } else if (ch == 2) {
            if (customers.empty()) cout << "Queue is empty\\n";
            else {
                cout << customers.front() << " is served\\n";
                customers.pop();
            }
        } else if (ch == 3) {
            if (customers.empty()) cout << "No waiting customers\\n";
            else {
                queue<string> copy = customers;
                while (!copy.empty()) {
                    cout << copy.front() << " ";
                    copy.pop();
                }
                cout << "\\n";
            }
        }
    } while (ch != 0);
    return 0;
}
```

Expected Output

```
1.Enqueue 2.Dequeue 3.Display 0.Exit
1
Customer name: Asha
Ticket booked
1.Enqueue 2.Dequeue 3.Display 0.Exit
1
Customer name: Rohan
Ticket booked
1.Enqueue 2.Dequeue 3.Display 0.Exit
3
Asha Rohan
1.Enqueue 2.Dequeue 3.Display 0.Exit
2
Asha is served
```

Assignment 9

C++ Code

```
#include <iostream>
#include <string>
using namespace std;

struct Node {
    string key, meaning;
    Node *left, *right;
};

Node* insert(Node* root, string key, string meaning) {
    if (root == nullptr) return new Node{key, meaning, nullptr, nullptr};
    if (key < root->key) root->left = insert(root->left, key, meaning);
    else if (key > root->key) root->right = insert(root->right, key, meaning);
    else root->meaning = meaning;
    return root;
}

Node* minimum(Node* root) {
    while (root->left != nullptr) root = root->left;
    return root;
}

Node* deleteKey(Node* root, string key) {
    if (root == nullptr) return root;
    if (key < root->key) root->left = deleteKey(root->left, key);
    else if (key > root->key) root->right = deleteKey(root->right, key);
    else {
        if (root->left == nullptr) {
            Node* temp = root->right;
            delete root;
            return temp;
        }
        if (root->right == nullptr) {
            Node* temp = root->left;
            delete root;
            return temp;
        }
        Node* temp = minimum(root->right);
        root->key = temp->key;
        root->meaning = temp->meaning;
        root->right = deleteKey(root->right, temp->key);
    }
    return root;
}

Node* search(Node* root, string key, int& comparisons) {
    while (root != nullptr) {
        comparisons++;
        if (key == root->key) return root;
        if (key < root->key) root = root->left;
        else root = root->right;
    }
    return nullptr;
}

void ascending(Node* root) {
    if (root == nullptr) return;
    ascending(root->left);
    cout << root->key << " : " << root->meaning << "\\n";
    ascending(root->right);
}

void descending(Node* root) {
    if (root == nullptr) return;
    descending(root->right);
    cout << root->key << " : " << root->meaning << "\\n";
    descending(root->left);
}

int main() {
    Node* root = nullptr;
    root = insert(root, "apple", "A fruit");
    root = insert(root, "book", "A set of pages");
    root = insert(root, "computer", "An electronic machine");
    root = insert(root, "dog", "A domestic animal");
    root = insert(root, "elephant", "A large animal");

    int ch, comparisons;
    string key, meaning;
    do {
        cout << "1.Add/Update 2.Delete 3.Search 4.Ascending 5.Descending 0.Exit\\n";
        cin >> ch;
        if (ch == 1) {
            cin >> key;
            getline(cin >> ws, meaning);
            root = insert(root, key, meaning);
        } else if (ch == 2) {
            cin >> key;
            root = deleteKey(root, key);
        } else if (ch == 3) {
            cin >> key;
```

```
            comparisons = 0;
            Node* found = search(root, key, comparisons);
            if (found) cout << found->meaning << "\\n";
            else cout << "Keyword not found\\n";
            cout << "Comparisons: " << comparisons << "\\n";
            cout << "Maximum comparisons in this BST: " << comparisons << "\\n";
        } else if (ch == 4) ascending(root);
        else if (ch == 5) descending(root);
    } while (ch != 0);
    return 0;
}
```

Expected Output

```
1.Add/Update 2.Delete 3.Search 4.Ascending 5.Descending 0.Exit
4
apple : A fruit
book : A set of pages
computer : An electronic machine
dog : A domestic animal
elephant : A large animal
1.Add/Update 2.Delete 3.Search 4.Ascending 5.Descending 0.Exit
3
dog
A domestic animal
Comparisons: 4
Maximum comparisons in this BST: 4
```

Assignment 10

C++ Code

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n, e, u, v;
    cout << "Enter number of locations and connections: ";
    cin >> n >> e;

    vector<vector<int>> matrix(n, vector<int>(n, 0));
    vector<vector<int>> adj(n);

    cout << "Enter connections (location numbers from 0):\\n";
    for (int i = 0; i < e; i++) {
        cin >> u >> v;
        matrix[u][v] = 1;
        matrix[v][u] = 1;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    cout << "Adjacency List:\\n";
    for (int i = 0; i < n; i++) {
        cout << i << " -> ";
        for (int x : adj[i]) cout << x << " ";
        cout << "\\n";
    }

    cout << "Adjacency Matrix:\\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) cout << matrix[i][j] << " ";
        cout << "\\n";
    }
    return 0;
}
```

Expected Output

```
Enter number of locations and connections: 5 5
Enter connections (location numbers from 0):
0 1
0 2
1 3
2 3
3 4
Adjacency List:
0 -> 1 2
1 -> 0 3
2 -> 0 3
3 -> 1 2 4
4 -> 3
Adjacency Matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 1 0
0 1 1 0 1
0 0 0 1 0
```

Assignment 11

C++ Code

```
#include <iostream>
#include <vector>
using namespace std;

void dfs(int vertex, vector<vector<int>>& graph, vector<bool>& visited) {
    visited[vertex] = true;
    cout << vertex << " ";
    for (int i = 0; i < graph.size(); i++) {
        if (graph[vertex][i] == 1 && !visited[i])
            dfs(i, graph, visited);
    }
}

int main() {
    int n, e, u, v, start;
    cout << "Enter number of locations and connections: ";
    cin >> n >> e;

    vector<vector<int>> graph(n, vector<int>(n, 0));
    for (int i = 0; i < e; i++) {
        cin >> u >> v;
        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    cout << "Enter starting location: ";
    cin >> start;

    vector<bool> visited(n, false);
    cout << "DFS Traversal: ";
    dfs(start, graph, visited);
    cout << "\\n";
    return 0;
}
```

Expected Output

```
Enter number of locations and connections: 5 5
0 1
0 2
1 3
2 3
3 4
Enter starting location: 0
DFS Traversal: 0 1 3 2 4
```